Changelog

What changed in each version.

Not released yet

Version 1.0.0 (6 October 2026)

The first full release. It has all the editors and tools of Fighter Factory Studio 3.7 for M.U.G.E.N with their options, and the emulator with its debugger. There are downloads for Windows, Linux and macOS. Not in it yet: the debugger for WinMUGEN (Debug > Start on Engine...).

Added

- Animations editor: the current frame is drawn over the timeline with its sprite, offset, flip, scale and angle, along with the player axis, the frame offset (secondary cross), the extra cross, rulers and guide lines. Dragging the sprite moves the frame offset. The panel edits the animation's number and name and the frame's sprite, offset, time, flip, transparency (with source and destination), loop start, scale, angle and interpolation, and can show an onion skin frame from any animation.
- Contact boxes: you draw them in a drawing mode that stays on until you right click. They can be selected, moved, resized by their handles, nudged with the arrow keys, deleted, switched between attack and collision and between regular and default, copied from the frame before, copied to every frame with the same sprite, and cut, copied and pasted. Default boxes show on the frames that use them, in the engine's default colors.
- Animations menu: New, Open, Add (with name, number and time), Delete, Duplicate (with a new number if you want), Add and Delete frame, Apply offset, time, flip, transparency and scale to all next frames, the Alignment Tool (offsets, box resizing and scaling for the animations you pick), Optimize, Play and Stop (60 ticks a second, looping at LoopStart, holding endless frames, with interpolation), Edit code (the whole file in the code editor, with the current animation and frame following the cursor), Edit animation code (one animation in its own window) and Save as animated GIF (scale and speed). The Sprite explorer adds frames to the current animation or makes new animations from a group's sprites.
- Every change made in the visual editor only rewrites the lines it touches, written the way Fighter Factory writes them, as one undo step in the text it shares with the code editor.
- Sprites > Switch to the frame's sprite: the Sprites editor shows the sprite of the current frame in the Animations editor.
- States > PalFX editor: shows a PalFX controller's values on the project's sprites (with the sine moving), loads templates from the palFX folder, and Insert puts it in the editor.
- Issues lists the lines in animation files that can't be read.
- Sounds editor: the current sound's waveform (one band per channel, a time ruler in ticks, the cursor and the selection) with six tools (Move view offset, Zoom, Free selection, Move selection, Horizontal and Vertical resizer). The panel has the sound selector, group, index, the sound's

format and length, the volume, the current sample and the selection. Waveform changes (Remove selected, Fill silence, Fade in, Fade out, Cut, Copy, Paste and the move and resize tools) wait until Save changes, which makes them one undo step. Discard changes throws them away, and leaving the sound or the editor with changes waiting asks first. Play, Play with loop, Pause and Stop start from the cursor, and the status bar shows the cursor, the selection and whether it's playing.

- Sounds menu: New, Open, Save as, Add (numbers from the file names or counting up), Delete, Duplicate, Change sound, Save sound as (WAV, OGG or FLAC), Resample (channels, rate and bits for the sounds you pick), Delete group, Rename group, Export (FFE and SndMaker lists with the WAV files) and Import (FFE, asking what to do with numbers already used).
- Palettes panel (on the right of the Sprites editor): the palette selector, Switch to the sprite palette, group and index, and the palette grid (select with click, Ctrl+click and Shift+click, double-click to edit a color with alpha, drag a color onto another slot). Palettes menu: Add (from palette files, or an empty 32-color palette), Delete, Remove all duplicates, Clear all unused, Clear, Change (ACT, PCX, PNG, GIF, BMP, PAL, ACO, ASE and Corel Painter swatches), Save as (ACT), Switch color count (32 and 256, and the colors come back), Exchange position of selected colors, Set the color of the selected slots, the Color picker, Apply to the sprites and the Remap tool. Palette color changes are undo steps of the sprite file, and Palettes > Undo and Redo reach them too.
- Advanced palette editor (Tools menu): the current sprite from the Sprites editor with its palette, next to a free image and palette, each with its own undo history and zoom. It can open and save palettes and images, add the palette to the sprite file, make a palette from images, put the palette into image files, make optimized and grayscale palettes and gradients, set colors, remove repeated colors, add the other palette's colors, fit the image to the palette, sort by tone, swap and invert colors, and apply twelve color effects. It also has hue, saturation and RGB sliders with a live preview, the mask, the color picker, sending colors and selections between the two palettes, and Commit changes and Commit sprite changes (with the palette handling dialog).
- Run in engine can also run projects in Ikemen GO, and offers its log folder after the run.
- Backgrounds editor (stages, screenpacks and storyboards): every element of the background group picked in the Backgrounds menu is drawn at the current tick the way the engine draws it (scrolling by delta, velocity, sine waves, tiling, parallax, 1.1 scales, animations and blending), with the screen edges, the players' start positions and the timeline. A click selects the element on top, dragging moves it, and the selection shows its tile and window. Play (with Pause and Stop) runs the group at 60 ticks a second inside the screen, and the arrow keys move the camera inside the stage's bounds.
- The element panel edits the name, type, sprite or animation, masking, start, delta, velocity, ID, layer, link to the element before, tiling and spacing, transparency (with source and destination), parallax (width or x scale), Y scale, 1.1 scale start and delta, the window and its delta, and both sine waves. The Controllers panel edits the controller definitions (name, loop time, up to ten element IDs) and their controllers (name, type, values, X and Y, time). Backgrounds menu: Add, Remove, Duplicate, Play, Pause, Stop and the Controllers commands. The Organizer hides elements and changes their order. Each change only rewrites the keys it changes, as one undo step in the definitions text it shares with the Definitions editor.

- Screenpacks also open their select.def in the Definitions editor.
- Project > New: the Project template window lists the template scripts of each category (scripts/project/<category>/<name>.lua, with a .png or .svg icon). The Empty template that comes with the program asks for a character, stage, font or storyboard and opens a blank project of that kind in the Definitions editor. Its files only live in memory, made from the engine's blank files (data/engines/mugen/new), have no name until they're saved and don't go in the history. Project > Save asks for a name for each changed file in turn ("Save definitions as..."), and saving the definitions puts the project on disk. New projects without a name are called "Project 1", "Project 2" and so on.
- Project template scripts: wizards and pages, radio buttons, check boxes, labels, line edits, combo boxes, spin boxes and layouts, message boxes, FF.translate, FF.emptyProject and FF.projectFromFile (also as a copy, whose files stay unnamed and changed until saved), and each engine's lists of values (MUGEN.ProjectType).
- Project > Open from archive: opens the project in a ZIP file (its first definitions file), unpacked into a folder of its own that is removed when the project closes. Saving one of its files asks where, starting in the archive's folder. Nothing is written into the archive. The history remembers the archive.
- Project > Open single file: each file you pick opens as its own project ("Project 1" and so on) in the editor for its kind, saves where it is and stays out of the history.
- Project > Save as: the whole project, with your changes, is written to the folder you pick with the definitions file renamed, and you go on working from there.
- Project > Create backup writes a ZIP of the project's files, with your changes, into a "backup" folder next to it ("04.10.2026_10.32.40_pm_.zip"). Options > General > Make a backup before loading a project writes one ("..._Auto.zip") every time a project is opened. Project > Load backup lists the backups in the Restore backup window, writes the one you pick over the project's files and then loads the project again.
- Options > General > Auto save: every few minutes, each changed file that has a name is saved where it is, quietly, and keeps its undo history.
- File watcher: a project file changed by another program is loaded again, asking first or not as Options > Messages > Reload watched files on modification says ("File modified"). Saves made by the program itself aren't reported.
- View > Organizer: rows can be dragged to a new place in every editor that has it (one undo step each).
- View > Organizer in the Animations, Sounds and Commands editors: their animations ("0 - Standing"), sounds ("0, 1") and commands (by name) move one, ten or all the way up or down, one undo step each. Organize automatically sorts animations by number and sounds by group and number. A double-click shows the item.
- Animations and Sounds > Import from another project: brings in the animations (with their comment line) or sounds of another open project, under new numbers picked in the list on the right, after the current one. For numbers already used it asks, replaces, skips or adds them anyway. Import sprites also brings the sprites the animations use that the project doesn't have.
- Tools > Export to HTML: a web page about a character or stage made from a layout (data/engines/mugen/layouts/<kind>/<name>/, with "!ff <key>!" placeholders). It has the details from the definitions, the e-mail, home page and comments you type in, the portrait, the standing

animation as a GIF for each palette, and the commands you pick drawn with button and direction pictures (made for layouts that don't have their own). For stages it has a still picture and an animated GIF of the backgrounds, at the size, frame rate and length you pick. The page uses the character set the layout asks for.

- Tools > Script editor: write, open, save and run Lua scripts. The toolbar has New, Open, Save, Save as, Run and Save for translation, and the code editor highlights Lua. Run shows what debugOutput() and print() write in the output box, and the errors in the message list (double-click to go to the line) and in a "Script error!" box. On the right, the engine and the script type pick the folder whose scripts are listed (double-click to open one). The window remembers its place and size.
- Tools menu scripts: every .lua file in scripts/tool (and in an engine's scripts/tool) becomes a Tools menu item that runs the script and its main(). A "Quick action script" you can change comes with it.
- The Lua API (see the Lua scripting document): the FF functions for projects (projectCount, projectAt, currentProject, currentEngine, engineList, engineByName), FF.emptyProject and FF.projectFromFile opening their project right away, the lists of values (FF.Filter, FF.FileType, FF.Mode, MUGEN.ProjectType with the same numbers as Fighter Factory and so on), and classes for projects and their files, code documents and cursors, sprites, palettes, animations with their frames and contact boxes, sounds, archives, files and widgets.
- Save for translation writes a Qt Linguist file with the texts of a script's FF.translate calls. A script's own translation (<name>_<language>.qm next to it) is used when it runs.
- Breakpoints: Debug > Toggle breakpoint (on the code editor's current line), or a click on the strip left of the line numbers, marks a line. Debug > Add breakpoint... opens the Breakpoint window (file and line, function, memory address or frame, with enabled, condition, ignore count and one shot). The Debugger panel lists them with their state icon, number, function, file, line, address, condition and ignore count, and can edit, turn on, turn off and remove them.
- Emulator: the project runs inside the program in an engine that works like M.U.G.E.N, using the engine folder it's in (or the one picked in the panel). A character fights itself or another character from the engine folder on its stage, played with the keys set in Options > MUGENext. A stage or fight screen project is shown with the engine's character, a storyboard plays its scenes, and a motif shows the system screen picked in the panel (title menu, versus, continue, victory, win, survival results). The match follows the engine's rules: states and controllers, triggers and expressions, physics, hits, guarding, throws, reversals, helpers, explods, projectiles, pauses, and rounds with intros, KO, time over and win poses. Changes you haven't saved in the editors are used.
- Emulator screen: the stage's layers, the players, effects and sparks in sprite priority order with the engine's palettes (an SFF v1 character's pal1, RemapPal), palette effects (PalFX, AllPalFX, BGPalFX), after-images, transparency and screen effects. Also the fight screen from fight.def (lifebars, power bars, faces, names, timer, combo counters, win icons and the Round, Fight, KO and winner calls in its fonts), the KO slowdown, and View > Attack box and Collision box over the game. The mouse wheel zooms. Sounds play with the engine's channels and panning, and the announcer's from fight.snd.
- Emulator panel: Versus or Training, Self or Random enemy, rounds, round time, explod mode (M.U.G.E.N 1.0 or 1.1 rules, or by each character's version), debug info under the players, and the

training options (dummy control Cooperative, AI with its level, or Manual; stance; auto guard; distance; button jam), which work while the match is running. Training fills life and power back up and nobody gets knocked out.

- Emulator debugging: every match is a debug session. It stops at breakpoints on [State] and trigger lines (with conditions, ignore counts and one shot), at frame and data breakpoints, and on Debug > Interrupt. Step into, over, out and frame go on from there. The line it stopped on opens with the current line marker. The Debug panel shows the values of the match and of each player (life, power, position, velocity, states, var, fvar and more), which can be watched in the Watch lists or get data breakpoints. The Debugger panel shows the call stack.
- Toolbars stay where they were dragged to (top, bottom, left or right, first or second row) the next time the program starts.
- Background tasks: loading and saving big sprite and sound files no longer freezes the window, and the progress shows in the status bar (click it to list the running tasks). Closing the program or running in the engine waits for them. Auto save can save in the background.
- Run in engine plays a character, stage or motif that's already in the engine folder right where it is, without a temporary copy (after saving it).
- Quick search in the graphic editors: find a sprite or sound by "group,number", or an animation by number or name.
- Help > Reference Documentation opens the program's own guide (editors, emulator, tools, shortcuts) in the browser.
- Downloads: a Windows installer that doesn't need admin rights and a portable ZIP, a Linux AppImage and a macOS disk image.
- Sprites > Change image asks whether to crop the new image's see-through border, which palette it gets (the Palette window), and for a linked sprite whether every copy gets it. Save image changes asks the last question too. Options > Messages remembers the answers.
- Options > Messages > Can't save empty sprite list, Can't save 8-bit sprites without palette and Can't save empty sound list stop those saves. When they're unticked, sprites without a palette can get a gray palette before saving.
- Options > Graphics: Invert sprite axis (the Sprites editor's axis boxes show the image's offset from the axis), Contact box over axis and Colors fit available space (the Advanced palette editor's grids fill the window's width).
- Project > Open from archive opens RAR files (RAR 4 and 5) as well as ZIP files, through the RAR extension, and the Archive class in scripts reads them too. Extension plugins can add other formats.
- Options > Graphics > Renderer: with Automatic or OpenGL, the Sprites, Animations and Backgrounds workspaces, the image editor and the emulator are drawn with OpenGL (DirectX also uses OpenGL, since Qt 6 has no ANGLE). Native painter draws them in software. If the OpenGL driver crashed before the first frame, the next start uses the native painter. The status bar shows which renderer is used.
- Windows: each project tab gets its own preview in the taskbar, showing the character's big portrait. Clicking it shows the tab, and closing it closes the project.
- The program can be translated. Qt's lupdate collects every menu, action, panel and message text (see the Contributing document).

Fixed

- A motif's select and fight files are checked, highlighted and completed as select and fight files. Before, they were treated as part of the system file, and Issues listed their blocks as unknown.
- On Windows, saving no longer fails when another program (an antivirus or an indexer) is reading the file at that moment. The file is written again a few times before an error is shown.

Fighter Factory bugs fixed here

- Emulator: palette and screen effects last their full time in frames (a PalFX with time = 1 shows for one frame).
- Saving sprites without a palette says how many there are (Fighter Factory showed "%n").
- A sound file with a sound that has no samples can be saved (Fighter Factory crashed).
- Animation numbers used twice are reported in Issues, and Duplicate only takes a number as the new number.
- An animation keeps one LoopStart, deleting a contact box numbers the rest from 0 again, Switch Contact box type keeps the box selected, and Optimize only turns repeated boxes into default boxes where the later frames keep the boxes they had, without leaving runs of empty lines.
- The Alignment tool's Scale Contact boxes never makes a box narrower than one unit. GIF delays add up exactly, the GIF shows the 1.1 scale and angle and keeps transparency for true color sprites, and the save dialog starts in the project folder.
- The cursor position is rounded down, the onion skin label follows Find animation, typing in text mode keeps the frame under the cursor, and the animation code window opens on the screen and asks before throwing away changes that weren't applied.
- Sounds: positive samples are drawn above the middle line, Fade in and Fade out work, and Stop and the end of a sound put the cursor back at the start. Numbers used twice are reported in Issues.
- Palettes: Delete and Remove all duplicates move the sprites to the palette that's left (Fighter Factory left them without one), and Delete asks when the palette is used. Clear all unused no longer skips the palette after each one it removes, and Clear asks first. Dragging a color onto another slot can be undone, the outline of the selected slot shows on every color, and the Save as dialog is titled "Save palette". Clearing the Remap tool draws the sprite again right away and its columns fit. 8-bit sprites without a palette and palette numbers used twice are reported in Issues.
- Backgrounds: transparency changes keep the "trans" key, and parallax modes write "width" or "xscale" (Fighter Factory wrote "default" and "paralwidth" and reset the values). Changing the type keeps the sprite, position, tiling and blending. Renaming a controller definition or controller keeps its kind of section (Fighter Factory turned them into elements). Unused controller IDs are left out (Fighter Factory turned them into 0), and an unticked X or Y removes its key (Fighter Factory wrote "nan"). An end time equal to the start is written, Add definition works for a group without definitions, and changing a window into a mask window takes away half the screen width. Element names have no space at the end, and the players' start crosses are only drawn for stages. Visible controllers work from their time on, velocities move one unit per tick and the sine offset is a phase, like in the engine. Closing a project after editing backgrounds no longer crashes.

- Storyboards open without a Sounds editor, the way Fighter Factory shows them.
- Advanced palette editor: Sort colors by tones and Remove the duplicate colors keep the image looking the same (the repeated colors get colors of their own, where Fighter Factory gave them one shared color). Invert all colors inverts the colors (Fighter Factory turned their order around). The optimized palette keeps the image drawn, and Insert the current palette in the images asks before writing over the files. Add to sprites editor no longer lists the new palette twice. Switching the palette or the sprite, or closing the window, asks before throwing away changes that weren't committed.
- Project > New: the blank character's sizes and speeds fit its 320x240 size (Fighter Factory's were four times too big). Save suggests each new file under the name the definitions give it, in the folder the definitions were saved to, and a file saved under another name gets that name in the definitions (Fighter Factory suggested no name and left the definitions as they were, so a saved project couldn't find its files). A double-click on a template runs it, and the window title shows the project's name for projects without one.
- Archives: folders written with backslashes (Windows PowerShell's Compress-Archive) open, an archive without a project says so, and archives are told apart by their own path (Fighter Factory refused a second archive with a file of the same name).
- Project > Save as writes every file of the project (Fighter Factory only wrote the definitions file, which then pointed at files that weren't there, and could crash later). Single files and archive projects can be saved too (Fighter Factory crashed).
- Open single file reports the files no editor can open (Fighter Factory did nothing) and keeps a file's byte order mark when saving.
- Backups: the definitions file is stored whole and every file under its own name and folder (Fighter Factory stored a broken definitions file and gave changed files full paths as names). Load backup lists the ZIP backups it makes and restores the one you pick (Fighter Factory only listed folders and restored nothing, and threw away unsaved changes). The list is in date order and shows am or pm. A project opened from an archive backs up all its files, and a new project that was never saved can't be backed up (Fighter Factory wrote to the root of the drive).
- The file watcher asks before loading a file with unsaved changes again, even when it's set to always reload (Fighter Factory threw the changes away without a word).
- Organizer: moving an animation keeps the empty lines between the actions where they were (Fighter Factory left actions without the empty line between them). Moving commands to the top or ten places down works (Fighter Factory crashed), a double-click on a command shows it in the code, and the sound file doesn't get a signature written into it.
- History: it's written as soon as it changes (Fighter Factory only wrote it on exit, so it was lost when the program didn't close normally). A recent project that doesn't exist anymore offers to leave the history, from Open recent and from the Welcome page.
- Import from another project: No in the question skips the item (Fighter Factory added a copy), and the summary counts the replaced items (Fighter Factory said none were imported).
- Export to HTML: the portrait tag has its size and is closed properly, "Author" is spelled right, every image tag is closed, the comments you type in are escaped, and the page is suggested under the project's name.
- The engine is called MUGENext everywhere (Options, the run configuration, file filters, the project template list), the way Fighter Factory shows it.

- Script editor: New, Open, closing and Esc ask before throwing away unsaved changes (Fighter Factory threw them away without a word). The title shows the file and whether it changed, and every toolbar button has a tooltip. Save for translation works and its message is worded right (Fighter Factory always refused it). print() output is shown (Fighter Factory showed it nowhere), and errors inside main() are reported (Fighter Factory hid them). A script that never ends can be stopped with Run or Esc while the window keeps working (Fighter Factory froze for good).
- Scripts: wrong calls give Lua errors where Fighter Factory crashed (widgets without a parent, methods called with "." instead of ":", Sprites:find, Archive:openFile and entries with plain tables, Palette.colorTable and the code node fields). Colors, positions, rectangles, property values and sounds come back as values you can read (Fighter Factory gave back objects you couldn't use). Folder archives take paths relative to the folder and have a name and path, and Archive's absoluteFilePath, absoluteFileName and isReadOnly are functions, like the documentation says.
- Breakpoints: turning one on or off doesn't mark the file as changed (Fighter Factory asked to save the file on exit). The Breakpoint window's OK button waits for the fields the type needs (Fighter Factory took an empty breakpoint and then dropped it), Browse... is only for file breakpoints, and its file filter is written right.

Version 0.5.0 (23 August 2026)

The Sprites editor is done (milestone M5): the sprite workspace and panel, every command in the Sprites menu, the image editor and the Sprite explorer.

Added

- Sprites editor (align mode): the graphic workspace (zoom levels, rulers, axis cross, guide lines, backgrounds, and dragging the sprite to move its axis) and the left panel (sprite selector, name, group, index, axis, 1 px moves, decompress on load, transparency color, onion skin). Every change can be undone, and axis changes in a row merge into one. Sprites menu: Add (the Add sprites dialog with numbers from the file names, axis points, palette modes, cropping and sharing), Delete, Duplicate, Change image, Save image, Save image (aligned) with axis.txt, Resize (six filters, and it can scale the frames' collision boxes too), Crop image, Delete group, Rename group, Alignment tool, Split channels, Export (FFE and SprMaker2 lists with PNG and ACT files), Import (FFE), and Save as in the M.U.G.E.N Beta, 1.0 or 1.1 format, turning true color sprites into 8-bit. The organizer moves and sorts sprites and palettes, numbers used twice are reported in Issues, and the status bar shows the cursor position, guide lines and SFF version.
- Sprite files keep linked sprites and palettes through any change, and are written back byte for byte when nothing changed.
- Image editor (Sprites > Image editor): the current sprite on a canvas you can zoom up to 32 times. It has layers (show and hide, rename, blend modes and opacity on RGBA canvases, Add, Delete, Duplicate, Merge visible, Merge all, Play), ten tools with their options bar (Move canvas, Move selection, Rectangle and Free selection, Magic wand, Color picker, Pen, Eraser, Remap pen with 32 color slots, Paint bucket), foreground and background colors, the Select, Image (Original, Properties) and Rotation (Rotate with four filters, flips) menus, its own undo history, and Copy and Paste. Save image changes writes the visible layers into the sprite and moves its axis with the canvas. Save visible as sprites writes each layer named "group,index" into that sprite. Leaving a sprite, the image editor or the Sprites editor with unsaved changes asks first.

- Sprite explorer (image editor mode): shows the previews of a group, which can be added as layers at the canvas origin or lined up on the sprite's axis, or saved as image files.
- Sprites > Import from another project: brings in sprites from another open project with new numbers, with palette handling and a choice of what to do with numbers already used. One undo step.
- Sprites > New and Open: an empty sprite file of the project's SFF version, or another SFF, in a new file tab.
- Edit > Cut, Copy and Paste of sprites: the sprite with its numbers, axis and palette, also as a picture for other programs. Pictures from other programs are pasted as new sprites.

Fighter Factory bugs fixed here

- Split channels gives each channel sprite its own channel, Add's Force and Exchange palette modes and "Crop before axis" work, Decompress on load is saved, Crop image treats index 0 as see-through for every range, and the status bar shows both guide positions.
- Image editor: point tools work on the pixel under the mouse (Fighter Factory used the nearest grid line). Rectangles dragged up and to the left select exactly what's between the grid lines, Move selection only fills the pixels it leaves, and the Magic wand works on palette index 1. Undoing Delete layer or a merge puts the layers back where they were, the blend mode follows undo, layers added from the explorer leave the current layer selected, turns by right angles are exact, and changing to 8-bit keeps see-through pixels see-through.
- Save visible as sprites makes the sprites that don't exist yet and keeps the editor's layers (Fighter Factory dropped both). Layers lined up from the explorer stay lined up after the canvas changed, and the explorer closes with the image editor.
- Import from another project only counts the new sprites.

Version 0.4.0 (15 August 2026)

The Definitions, Commands and States editors are done (milestone M4), with Run in engine, the variable usage list and log, the Throw creator and the Offset viewer.

Added

- Audio library (ofsaudio, without Qt): sample buffers with WAV conversion, reading WAV (PCM, float, A-law, mu-law, IMA and Microsoft ADPCM, MPEG), Ogg Vorbis, FLAC and MP3 (recognized by their content), writing WAV, Ogg Vorbis and FLAC, resampling, the waveform editing of the Sounds editor, and waveform peak summaries. Sound playback with a cursor, pause and loops, using miniaudio (AudioEngine, SoundPlayer).
- Project > Run in engine: the project is copied into the engine folder of the run configuration you pick (characters to chars/ofs_N, stages to stages/ofs_N.def, screenpacks to data/ofs_N) with your unsaved changes. The engine is started there, its mugen.log shows in Engine Output when it closes, and the copy is removed. Files that were already in the engine folder are never written over or deleted. Run configurations are saved in mugen_config.xml, the same way Fighter Factory saves them.
- States > View variable usage list: the code block tree shows, for each of the 60 integer and 40 float variables, the StateDefs that set it (click one to show it at the top of the editor). It follows your changes and remembers which entries you opened. Float variables and fvar(n) :=

assignments are listed as floats, and system variables are left out (Fighter Factory mixed them up).
- States > Insert variable usage log: the same list as a comment block at the cursor, by StateDef number, name or both, with every slot or one StateDef per line if you want.
- States > Throw creator: makes the states of a throw (attempt, throw with one TargetBind per element, held and thrown opponent) from the project's animations, shows both players on each element of the throw (the opponent drawn with the sprites of any open project, showing the thrown animation's frame at that moment), and puts the code in at the cursor. Add animation, Insert frame, Remove current frame and the Group, Index and flip fields edit the thrown animation in the AIR file directly. The text comes from data/engines/mugen/scripts/throw.lua, which you can edit. Different from Fighter Factory, the attempt uses the Start animation and the throw uses the Throwing animation, Insert frame splits at the element shown, Add animation uses its Time and refuses numbers already used, and the preview stays visible at big screen sizes.
- States > Offset viewer: a frame from one of the project's animations next to a frame from an animation of any open project, moved by an X and Y offset and flipped if you want, to work out bind and helper offsets. The viewer doesn't change anything in the project.
- New, Open and Save as in the Definitions, Commands and States editors (and the general Editor actions). New starts a file without a name in a new file tab that asks for a name when you save it, Open adds a file as a file tab starting in the editor's last folder, and Save as saves the current file under a new name.
- Engine plugins can run Lua scripts that return text (Script::callFunction) and open their own tool dialogs from their actions (Engine::toolActions, Engine::runTool).

Fixed

- Save as: the file tab, its checks, its code tree and the project's file list now follow the file to its new name (they lost track of it before).

Version 0.3.0 (1 August 2026)

The first preview: the main window, the file formats library and the code editors (milestones M0 to M3).

Added

- Project setup: CMake build with presets for Windows (llvm-mingw, MSVC), Linux and macOS.
- The core library (ofscore), the file formats library (ofsformats), the shared Lua library (ofslua) and the M.U.G.E.N engine plugin.
- Engine plugins are found and loaded, with a version check.
- Unit and GUI tests (Catch2, QtTest) and automatic builds.
- Settings files (config.xml, shortcuts.xml, history.xml) with defaults and user changes.
- All the actions of the program and the M.U.G.E.N editors, with shortcuts for each editor and menu items that show their keys.
- Lua presets for the interface (Classic, Clean, Neo, Ultimate) that build the toolbars and editor panels.

- Main window: menus, Main and Extra toolbars, project tabs, the welcome page with recent projects, left and right panels, file tabs, bottom panels and the status bar. M.U.G.E.N projects of every type open.
- Options dialog (General, Interface with the keyboard shortcut editor, Messages, Colors, Graphics, Code and the M.U.G.E.N tab), About dialog, Calculator and Clipboard container. Settings can be exported to one file, settings from an older install can be imported, and default settings come with the program.
- Themes: the Dark (default), Light, Classic and Ultimate interface styles, the six color sets, and the Nuvola, Flat-Dark, Flat-Light, Classic and Ultimate icon sets with their M.U.G.E.N extras, made from openly licensed icons by tools/iconsets/build_iconsets.py.
- File formats library (ofsformats, without Qt):
    - SFF sprite files v1, v2.0 and v2.01: reading, writing, saving files that weren't changed byte for byte, changing between v1 and v2, linked sprites and palettes, and choosing the compression for each sprite.
    - Sprite codecs RLE8, RLE5, LZ5, PCX and PNG (with exact palette indices).
    - SND sound files, WAV files (PCM 8, 16, 24 and 32-bit and float, unknown chunks kept) and FNT v1 bitmap fonts.
    - Palette files: ACT (M.U.G.E.N's backwards order and the normal one), RIFF PAL, JASC-PAL, ACO (reading) and palettes from PCX and PNG images.
    - Image tools: crop to the content, flips, nearest and bilinear resize, Scale2x, and fitting RGBA images to a palette.
- Images, palettes and tool input files for the editors (ofsformats): reading and writing GIF (all frames, local palettes, transparency, disposal, delays, looping), reading and writing BMP (1, 4 and 8-bit with RLE, 16, 24 and 32-bit), ASE palettes, writing ACO, palettes from GIF and BMP images, the six filters of the Resize dialog (nearest neighbor, bilinear, EPX/Scale3x and hqx through the hqx library), and reading, writing and exporting SprMaker2 .def files and SndMaker lists from SFF and SND files, checked against Elecbyte's sprmake2 and sndmaker.
- ofs-sffdump, a command line tool to look inside SFF files, extract and convert them, and list SND files.
- The byte layouts and odd cases of every format, in the File formats document.
- Tests on real M.U.G.E.N files on your computer and checks against Elecbyte's sff2png and sprmake2 (optional, turned on with OFS_CORPUS_KFM and OFS_CORPUS_MUGEN).
- M.U.G.E.N language data for the code editors: the syntax database (every file type, state controller and trigger from the DOS versions to 1.1), completion lists, snippets, state controller templates, PalFX presets and reference pages for controllers and triggers, with a checker and tests on real files.
- Code editors: line numbers, folding sections, highlighting of the current line, block and brackets, syntax highlighting from the syntax database, comment on and off, zoom, completion lists, snippets with input fields, and reference pages for controllers and triggers on Ctrl+click.
- Definitions, commands and states are checked as you type. Problems are underlined, listed in the code toolbar and the Issues panel, counted in the status bar, and the previous and next error and deprecation commands jump to them. ofs-validate runs the same checks from the command line.

- Text encodings: the file's encoding is detected, Edit > Encoding (also in the status bar), the default charset option, and changing tabs to spaces and back on open and save.
- Search: the quick search bar (case, regular expressions with %1 captures, backwards and wrap around), the Find and Replace dialog for several lines, and Go to line.
- Advanced Search in all projects, the current project, the current file, the open files or a folder, with file and exclude patterns, results grouped by file, Search & Replace with check boxes, and the Lazy syntax fix (keywords written with their first letters are spelled out).
- Edit > Code > Auto-indent selection and Auto-format selection (lined up = signs and comments, respaced expressions), and the code editor's right click menu with the Code submenu.
- Code snippet editor (fields, preview, save, save for translation) and the code block trees of the Definitions, Commands and States editors, with Current line navigation.
- States editor: the list of state controller templates with New, Edit and Delete.
- Project explorer: a double-click opens a file in its editor.